ASIGNATURA · ARQUITECTURA DE SOFTWARE

Arquitectura de Software

Fundamentos, atributos de calidad, estilos y documentación



AGENDA

Lo que veremos hoy

01	**Fundamentos**	Definición, importancia y rol del arquitecto
02	**Atributos de calidad**	Qué hace “bueno” a un sistema y sus compromisos
03	**Estilos arquitectónicos**	Capas, microservicios, eventos y más
04	**Documentar la arquitectura**	Vistas 4+1, modelo C4 y registros de decisión (ADR)
05	**Proceso y cierre**	Ciclo de diseño, conclusiones y actividad práctica

FUNDAMENTOS

¿Qué es la arquitectura de software?

> Es el conjunto de estructuras que permiten razonar sobre un sistema: sus elementos, las relaciones entre ellos y las propiedades de ambos.
>
> Adaptado de Bass, Clements y Kazman, *Software Architecture in Practice*.



Elementos
Módulos, componentes, servicios



Relaciones
Interfaces, conectores, dependencias



Propiedades
Rendimiento, seguridad, disponibilidad

FUNDAMENTOS

¿Por qué importa?

La arquitectura concentra las decisiones más difíciles y costosas de cambiar después.

Comunica

Ofrece un lenguaje común entre desarrolladores, clientes y gerencia para discutir el sistema.

Anticipa

Fija temprano las restricciones que determinan rendimiento, seguridad y costo.

Reutiliza

Un modelo probado puede transferirse a otros sistemas con requisitos similares.

FUNDAMENTOS

El rol del arquitecto de software

01

Prioriza la calidad

Identifica los atributos de calidad que más importan al negocio.

02

Decide y documenta

Elige estilos y tecnologías, y deja registrado el porqué.

03

Equilibra compromisos

Sopesa costo, tiempo y riesgo frente a cada alternativa.

04

Comunica

Traduce entre lo técnico y lo de negocio con todos los interesados.

05

Guía al equipo

Acompaña la implementación para que respete el diseño acordado.

06

Gestiona la deuda técnica

Vigila la erosión del diseño y planea su corrección.

ATRIBUTOS DE CALIDAD

Qué hace “bueno” a un sistema

Atributo	Pregunta que responde	Táctica típica
Rendimiento	¿Qué tan rápido responde bajo carga?	Caché, concurrencia
Disponibilidad	¿Sigue operando ante fallos?	Redundancia, heartbeat
Seguridad	¿Resiste accesos no autorizados?	Autenticación, cifrado
Modificabilidad	¿Cuánto cuesta cambiarlo?	Bajo acoplamiento
Escalabilidad	¿Crece al aumentar la demanda?	Escalado horizontal
Usabilidad	¿Es fácil de aprender y usar?	Separar la interfaz

IDEA CLAVE

No existe la arquitectura perfecta: solo compromisos bien razonados.

Más seguridad

Cifrado y validaciones extra suelen costar rendimiento.

Más disponibilidad

Réplicas y redundancia elevan el costo y la complejidad.

Más modificabilidad

Capas de abstracción añaden indirección y latencia.

ESTILOS ARQUITECTÓNICOS

Patrones para organizar un sistema

Capas

Niveles con responsabilidades separadas, cada uno sobre el anterior.

Ej.: aplicaciones empresariales

Cliente-servidor

Los clientes solicitan servicios a un servidor central.

Ej.: aplicaciones web

MVC

Separa modelo, vista y controlador para aislar la interfaz.

Ej.: Django, Spring MVC

Microservicios

Servicios pequeños e independientes, desplegables por separado.

Ej.: plataformas de streaming

Orientada a eventos

Componentes que reaccionan a eventos de forma asíncrona.

Ej.: pagos, IoT

Tuberías y filtros

Los datos fluyen por etapas de transformación encadenadas.

Ej.: compiladores, procesos ETL

ESTILOS ARQUITECTÓNICOS

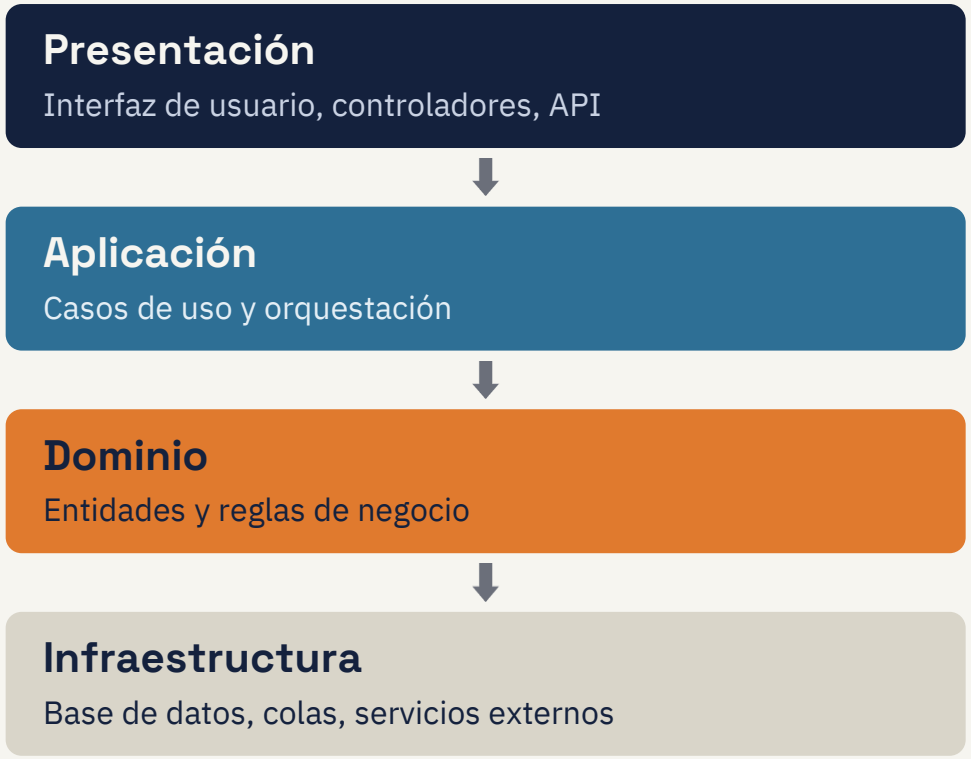
Monolito vs. microservicios

Criterio	Monolito	Microservicios
Despliegue	Una sola unidad	Cada servicio por separado
Escalabilidad	Se escala todo el sistema	Se escala solo lo necesario
Complejidad operativa	Baja	Alta: red, monitoreo, orquestación
Datos	Una base compartida	Una base por servicio
Equipos	Un equipo coordinado	Equipos autónomos por dominio
Conviene cuando	Producto nuevo o equipo pequeño	Sistema grande y dominios claros

Consejo: muchos equipos empiezan con un **monolito modular** y extraen servicios cuando el dominio madura.

ESTILOS ARQUITECTÓNICOS

Ejemplo: arquitectura en capas



Dependencias hacia abajo

Cada capa solo usa los servicios de la capa inmediatamente inferior.

Responsabilidades separadas

Cambiar la interfaz no obliga a tocar las reglas de negocio.

Riesgo a vigilar

Capas que solo reenvían llamadas añaden latencia sin aportar valor.

DOCUMENTACIÓN

Modelo de vistas 4+1 (Kruchten)

Vista lógica

Para: usuarios finales

Qué hace el sistema: clases, módulos y sus relaciones.

Vista de procesos

Para: integradores

Concurrencia, comunicación y rendimiento en ejecución.

Escenarios

+1

Casos de uso que unen y validan las cuatro vistas.

Vista de desarrollo

Para: programadores

Cómo se organiza el código: paquetes, capas, librerías.

Vista física

Para: infraestructura

Despliegue del software en servidores, nodos y redes.

DOCUMENTACIÓN

Modelo C4: zoom a la arquitectura

Nivel 1

Contexto

El sistema como una caja, con sus usuarios y sistemas externos.

Para: todos

Nivel 2

Contenedores

Aplicaciones, bases de datos y servicios desplegables.

Para: equipo técnico

Nivel 3

Componentes

Partes internas de cada contenedor y sus responsabilidades.

Para: desarrolladores

Nivel 4

Código

Clases e interfaces; suele generarse desde el IDE.

Para: quien implementa

Propuesto por Simon Brown: cada nivel es un mapa con más detalle que el anterior, como acercarse en un mapa digital.

DOCUMENTACIÓN

Registrar decisiones: los ADR

Un **Architecture Decision Record** es un documento breve que captura una decisión importante, su contexto y sus consecuencias.

- Se guarda junto al código, por ejemplo en /docs/adr
- Es inmutable: si la decisión cambia, se escribe un ADR nuevo
- Explica el porqué a quien llegue después

EJEMPLO

ADR-007 · Usar PostgreSQL

Estado: Aceptada

Contexto: Se requieren transacciones ACID y consultas relacionales complejas.

Decisión: Adoptar PostgreSQL como base de datos principal.

Consecuencias: + madurez y soporte de JSON; - el equipo necesita capacitación.

PROCESO

Ciclo de diseño arquitectónico

1

Analizar

Recoger requisitos y priorizar atributos de calidad con escenarios.

2

Diseñar

Elegir estilos, patrones y tácticas que satisfagan esos atributos.

3

Documentar

Describir vistas (4+1, C4) y registrar decisiones (ADR).

4

Evaluar

Revisar riesgos y compromisos, por ejemplo con el método ATAM.

5

Evolucionar

Implementar, medir y ajustar la arquitectura con el tiempo.

Es un ciclo iterativo: lo que se aprende al evaluar e implementar vuelve a alimentar el diseño.

CIERRE

Conclusiones

1 **La arquitectura es decisión**

Define estructuras difíciles de cambiar; merece pensarse con cuidado.

2 **La calidad guía el diseño**

Los atributos de calidad, no la moda tecnológica, deben orientar las elecciones.

3 **Todo es un compromiso**

Cada estilo gana en unos atributos y cede en otros.

4 **Documentar es comunicar**

Vistas, C4 y ADR hacen que el conocimiento sobreviva al equipo.

ACTIVIDAD EN CLASE

Diseñemos una app de domicilios

Caso RapiPedido: una empresa emergente lanzará una plataforma de domicilios en una ciudad grande. Ustedes son su equipo de arquitectura.

Clientes

Buscan restaurantes, pagan en línea y siguen su pedido en vivo.

Restaurantes

Reciben pedidos, actualizan su menú y avisan cuando está listo.

Repartidores

Aceptan entregas, comparten su ubicación y confirman la entrega.

Administración

Gestiona comisiones, soporte, reportes y disputas.

Organización: Grupos de 4 · 50 minutos · Roles: líder · analista de calidad · diagramador · relator

ACTIVIDAD EN CLASE

El caso: requisitos y supuestos

Requisitos funcionales

- Buscar restaurantes y platos por zona
- Armar el carrito y pagar en línea
- Asignar un repartidor automáticamente
- Seguir el pedido en tiempo real en el mapa
- Notificar cada cambio de estado del pedido
- Calificar al restaurante y al repartidor
- Panel web para que el restaurante gestione su menú

Supuestos del caso

Aspecto	Valor
Clientes activos	200.000 al mes
Pico de pedidos	3.000 por hora (almuerzo y cena)
Ubicación del repartidor	Se envía cada 5 segundos
Pagos	Pasarela externa
Equipo	8 desarrolladores
Lanzamiento	En 6 meses

Cifras ficticias, definidas solo para el ejercicio.

ACTIVIDAD EN CLASE

Paso a paso y tiempos

Paso	Qué hacer	Entregable	Tiempo
1. Comprender	Leer el caso y listar actores, interacciones y sistemas externos.	Mapa de actores	5 min
2. Priorizar	Elegir 3 atributos de calidad y escribir un escenario medible para cada uno.	3 escenarios	10 min
3. Elegir	Comparar al menos dos estilos frente a los atributos priorizados.	Justificación	10 min
4. Diagramar	Dibujar el C4 nivel 2: apps, servicios, bases de datos y sistemas externos.	Diagrama C4	15 min
5. Registrar	Escribir un ADR con la decisión más importante del grupo.	1 ADR	10 min

Socialización: cada grupo presenta 3 minutos y responde 2 minutos de preguntas.

ACTIVIDAD EN CLASE

Plantilla: escenario de calidad

Ejemplo resuelto para **rendimiento en hora pico**:

1 · FUENTE
3.000 clientes simultáneos

2 · ESTÍMULO
Confirman y pagan su pedido

3 · ARTEFACTO
Servicio de pedidos

4 · ENTORNO
Operación normal, hora del almuerzo

5 · RESPUESTA
El pedido se registra y se notifica al restaurante

6 · MEDIDA
En menos de 2 s para el 95 % de los pedidos

Escriban uno así para cada atributo priorizado. Sin **medida**, no es un escenario: es un deseo.

ACTIVIDAD EN CLASE

Rúbrica y preguntas guía

Criterio	Peso	Lo que se espera
Atributos y escenarios	25 %	Tres atributos pertinentes, con escenarios medibles
Elección del estilo	20 %	Justificada en los atributos, no en preferencias
Diagrama C4 nivel 2	25 %	Contenedores, tecnologías y comunicaciones claros
ADR	20 %	Contexto, decisión y consecuencias a favor y en contra
Presentación	10 %	Claridad, tiempo respetado y respuesta a preguntas

⚠ ¿Qué pasa si la pasarela de pagos deja de responder?

¿Cómo llega al cliente la ubicación del repartidor en tiempo real?

¿Qué parte del sistema debe escalar en hora pico?

¿Preguntas?

Gracias por su atención.

LECTURAS RECOMENDADAS

Bass, Clements y Kazman. *Software Architecture in Practice.*

Richards y Ford. *Fundamentals of Software Architecture.*

Kruchten (1995). *The 4+1 View Model of Architecture.*

Simon Brown. *The C4 Model for Visualising Software Architecture.*